

Efficient Porting of FPGA Independent Testing Within and Across Vendors

Travis Haroldsen, Ting-Yuan Sung, and Osaze Shears

Information Sciences Institute
University of Southern California
{tharoldsen,tsung,shears}@isi.edu

Dallon Glick and Jay Danner

Georgia Tech Research Institute
Georgia Institute of Technology
{dallon.glick,jay.danner}@gatech.edu

Abstract—Critical systems in our infrastructure contain a wide range of aging FPGA devices in need of replacement. The variety of FPGAs, from multiple families and across 30+ years, presents a challenge to a general purpose tool for validating procured replacement devices to ensure correct and trusted execution of the platforms. CIFT addresses this challenge by providing common infrastructure for creating non-destructive stuck-at fault testing for FPGAs. With this infrastructure, CIFT has quickly added support for testing 14 different device families from multiple vendors. This paper addresses the techniques used to create a flexible testing infrastructure and explores new areas that CIFT is expanding into.

Keywords—FPGA; testing; validation; Xilinx; Microsemi; Intel

I. INTRODUCTION

Field Programmable Gate Array (FPGA) wear out is a problem in many aging, critical systems. In these systems, it is becoming essential to evaluate the health of the FPGAs components and if necessary replace them. However, simply swapping in new FPGAs can be challenging, because, in many situations, the FPGAs in these systems may no longer be manufactured. This requires replacement FPGAs to be procured from deep storage or from potentially untrusted third parties. In both cases, uncertainties about the histories and health of the parts require them to be validated as part of a trust and assurance process. The prevalence of counterfeits of these older FPGAs makes validating them even more important.

Independent FPGA functional testing remains an important first step in quantifying the health of fielded, stored and acquired chips and in establishing a trusted supply chain. In FPGA functional testing, the resources on a device are exercised to expose faults in the underlying VLSI and configuration memory. Unlike more extensive means of validation like high-resolution imaging, functional testing is low-cost, fast (minutes to hours per device) and non-destructive. This is accomplished by using specialized bitstreams which exercise the programmable logic and interconnect in the FPGAs and detects faults within the devices.

A significant challenge in independent functional testing for FPGAs is effectively supporting the breadth of FPGAs used in these systems including FPGAs from at least 4 vendors and

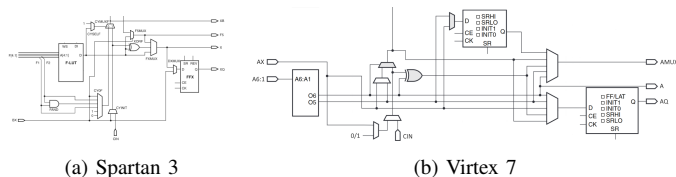


Fig. 1. Differences in generations of Xilinx slice structures

spanning over 30 years evolution. While all families of FPGAs have similarities, each is unique and requires a custom set of test cases to provide high test coverage. Additionally, each family is typically comprised of several parts and packages varying in size and resource selection each requiring its own set of test bitstreams.

The Comprehensive Independent Functional FPGA Testing (CIFT) tool addresses the breadth of FPGAs by providing an adaptable and scalable infrastructure for testing resources commonly found on FPGAs including the Programmable Logic Blocks (PLBs), embedded Block RAMs (BRAMs) and programmable routing interconnect. Built on top of this infrastructure, CIFT supports independent functional testing for most Xilinx FPGA devices up through UltraScale+ and Microsemi PolarFire and SmartFusion family devices. Work is also ongoing to support recent Intel FPGA families.

This paper describes the CIFT framework and heuristics we use to support a wide range of FPGA devices and to scale testing across device sizes. It describes how support for a new family is added to CIFT and possible uses. Lastly, it discusses ongoing research areas that CIFT is exploring.

This paper is organized as follows. Section II provides brief description of FPGA architecture and independent functional testing. Section III describes the testing infrastructure within CIFT and the process of porting to a new device. Section IV describes possible applications for CIFT. Finally, Section V discusses on going work and concludes the paper.

II. BACKGROUND

A. Independent Functional Testing

Independent functional testing is the functional validation of the logic and routing resources of a hardware device performed independent from the manufacturer of the device. While device vendors perform initial validation of devices at fabrication, this testing may be insufficient or inapplicable to

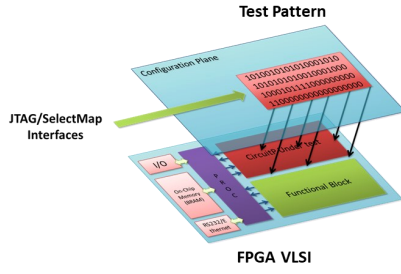


Fig. 2. FPGA Configuration Memory Controlling Circuit

user's needs. First, the testing is performed only once at the beginning of the device while users may need to perform testing potentially years after the production of the chip or while in the field. Second, the testing is proprietary and performed by the device manufacturer which prevents it from being independently audited and quantified, without which a users trust and assurance needs may not be met.

Unlike vendor testing, CIFT provides open testing that may be performed at any time by the user. CIFT tests take advantage of the inherent configurability of FPGAs to implement functional tests fully within the FPGA fabric by programming specially-crafted bitstreams onto the devices (Figure 2). These *test bitstreams* implement fully-contained test circuits including any overhead logic needed to test one or more components on the device. Multiple test bitstreams are used to test the full device. By using bitstreams to perform testing, testing can to be performed just by programming bitstreams to the device without any need for additional hardware. CIFT also imposes additional constraints on itself of disallowing the need for a custom test harness and minimizing IO requirements – most CIFT tests require access to the BSCAN as the only IO requirement range of

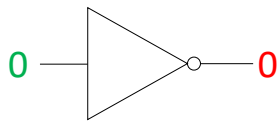


Fig. 3. Gate level view of a stuck-at-0 type fault

The primary CIFT tests focus on detecting stuck-at faults in a devices. Stuck-at faults are a fault mode where a wire is incapable of producing either a logic 1 or logic 0. Stuck-at faults are detected by applying a stimulus to a resource to produce a desired result on the output and then comparing by observing the actual response on the wire and comparing it to the expected value. A discrepancy between the expected and actual value on the output, such as in Figure 3, indicates a stuck-at fault in the circuit.

To test, CIFT uses a launch and capture technique where (compare Figure 4) test inputs generated by a *Test Pattern Generator* (TPG) drive the resource being tested, referred to as the *Unit Under Test* (UUT). The UUT response is then com-

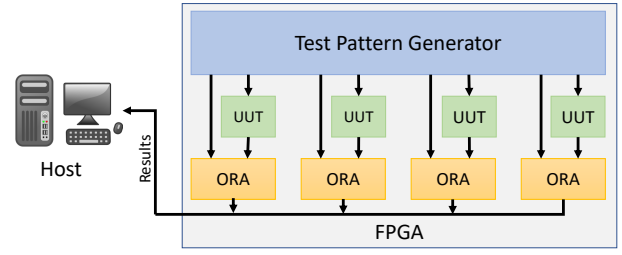


Fig. 4. General CIFT Test Architecture

pared against the expected output with the *Output Response Analyzer* (ORA). CIFT consolidates all responses into a single pass/fail result for the circuit though finer granularity tracking could be achieved. This result is then read by the host over the JTAG connection and used to in assessing the status of the device.

B. FPGA Architecture

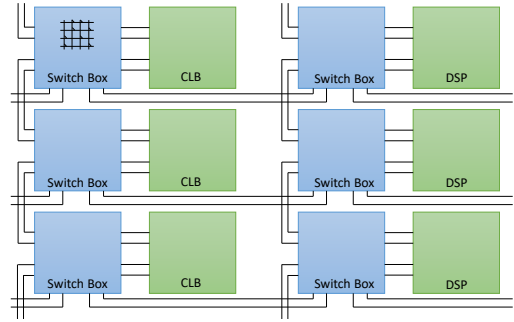


Fig. 5. Sea of gates FPGA architecture

Most FPGAs are implemented as a "sea of gates" architecture where the logic resources are loosely connected by a web of routing interconnect in the *switchboxes* (see Figure 5). The logic resources can be of many types such as on chip block RAMs (BRAM), multipliers, other hard IP, with the most common type being the programmable logic blocks (PLBs) or Slices and LABs in Xilinx and Intel nomenclature. PLBs, such as seen in Figures 1a and 1b, are composed of lookup tables (LUTs), flip-flops (FFs) and other logic that provide the FPGA their versatility.

Typically adjacent to the logic resources are the switchboxes that contain the programmable routing paths in the FPGAs. Each switchbox contains numerous wires going to other switchboxes or logic tiles and *programmable interconnect points* (PIPs) that optionally connect two wires.

The switchbox PIPs and the LUT contents and routing paths in the PLBs are controlled by the contents of the configuration memory. When programming a bitstream to a device, the contents of the bitstream are written to the configuration memory and used to enable or disable the corresponding resource to implement a design. Most FPGAs are reprogrammable allowing the configuration memory to be configured many times either through on chip reconfiguration circuitry or external interfaces such as JTAG.

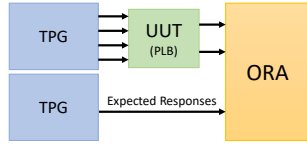


Fig. 6. CIFT PLB Test Case

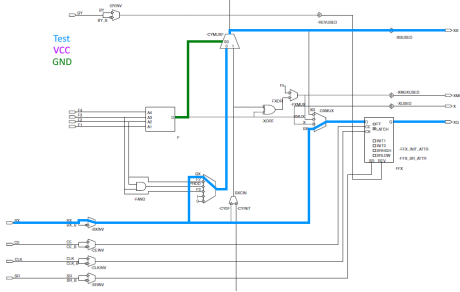


Fig. 7. An example UUT

III. TECHNICAL APPROACH

CIFT aims to provide testing for a wide variety of FPGA devices from multiple manufacturers. To accomplish this goal, CIFT includes a wide range of general infrastructure for testing Programmable Logic Blocks, Block RAMs and programmable routing interconnect. The infrastructure includes general test architecture utilities such as output response analyzers, TPGs for the common FPGA structures, algorithms for performing layout and partitioning of device resources, and interfaces for communicating with a host system. The infrastructure is highly applicable to many FPGA families and allows for quickly adapting CIFT to new device architectures by extending the utilities to target the unique characteristics of each family of FPGAs.

A. Programmable Logic Blocks

The programmable logic blocks are the most prevalent logic resource in most FPGA families and CIFT performs extensive testing on them. This testing includes thorough analysis of resources in the PLBs including:

- the lookup tables
- large input function muxes (e.g. F7/8/9 muxes in Xilinx devices)
- flip-flops including the enable and reset circuitry
- all tractable routing paths and muxes
- memory capabilities,
- arithmetic logic and carry chains

CIFT tests PLBs with the architecture shown in Figure 6. In this test, a TPG generates one or more input signals to the UUT. An expected response is generated by another TPG; the separate TPG is used to protect against a common source of failure leading to false negatives. The UUT is a single PLB configured to test a chosen set of resources and configurations in the PLB. The configuration of the UUT is dependent on the device and is usually created using highly constrained HDL to create the desired circuit. Within the PLB, resources and paths

may be mutually exclusive within a configuration, so multiple test cases, i.e. bitstreams, are created to test each resources.

CIFT provides configurable TPGs for targeting all of the common PLB components. For many resources, including the lookup tables, routing muxes and arithmetic logic, CIFT provides a flexible TPG that allows the user to specify a sequence of inputs to test the UUTs with. When compiled, this sequence is compiled into the logic for the test and will run through completion. For other resources such as the flip-flops and other memories, CIFT provides more focused TPGs such as a MATS TPG for testing LUTRAMs and a TPG to perform exhaustive testing of the flip-flops and their control circuitry.

To significantly reduce the time needed for testing, each CIFT test bitstream tests many PLBs in the device in parallel. However, as CIFT contains the TPG, ORA, and other logic – all which are implemented with the PLBs – within the bitstream, CIFT cannot test all PLBs in a single test bitstream. To partition the tests, CIFT has tools to identify all PLBs in a device from external device databases and creates a test layout that can adapt to different size parts within a family.

CIFT focuses on reducing the number of test bitstreams at the expense of fault resolution, i.e. multiple components in a PLB are tested within each test case and a fault in any will mark the entire PLB as failing for the test. In general, PLBs testing in CIFT requires around 20 test cases implemented with 50-100 bitstreams and can be tested over JTAG in a matter of minutes. Samples of test families are provided in Table I.

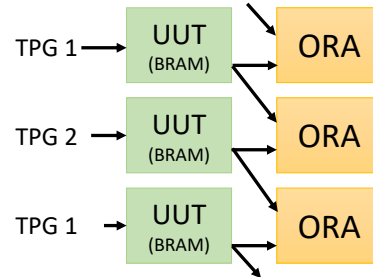


Fig. 8. CIFT BRAM Test Architecture

CIFT tests the on-chip block RAMs with conventional memory tests. Provided TPGs include the MarchX, MarchB, MarchLR, and MarchS2PF (for dual port RAM faults) march patterns which cover all stuck-at bit faults, faults in the address decode logic, many coupling faults in the memory cells, and faults in memory initialization. As BRAMs on FPGAs frequently support multiport configurations, each single port march pattern is adapted to support two and four port RAMs by repeating the test pattern on each port and are easily configurable to support different widths and depth. CIFT also provides a TPG providing basic functional coverage for built-in FIFO operations to the BRAMs common in FPGAs.

As with PLB tests, CIFT will create a layout for BRAM tests. Because the BRAMs are not used in the overhead

TABLE I
SAMPLE PLB TEST METRICS

Family	Device	# Tests	# Bitstreams	Test Time
Xilinx Kintex 7	xc7k70t	16	47	Measure
Microsemi SmartFusion	To choose	Numbers coming		Measure
Microsemi PolarFire	To choose	Numbers coming		Measure

logic, all BRAMs on a device can frequently be covered in a single bitstream. Nevertheless, CIFT does have partitioning support to allow the user to target a subset of BRAMs in each bitstream. The layout for BRAMs (Figure 8) differs from the PLB test architecture (contrast with Figure 6) with each UUT driving its own associated ORA and the ORA of a neighboring BRAM. The last BRAM connects to the ORA of the first BRAM completing a chain ensuring all BRAMs have equivalent outputs. Each neighboring UUT is driven by a separate TPG to reduce false negatives due to possible faults in the TPG logic.

CIFT BRAM tests cover all width/depth tradeoff configurations for the BRAMs as well as other common modes of operation (simple dual port, FIFO, cascaded-BRAMs for larger address space). The number of tests to cover all of these configurations is typically in the tens to twenties of bitstreams and can be performed on hardware in minutes. BRAM test cases and test time for sample devices are provided in Table II.

C. Routing

The routing interconnect makes up a sizable portion of the resources on an FPGA. For reference, the Xilinx Virtex 7 XC7X980T contains over 100 million programmable switches and, altogether, the routing fabric can account for more than 50% of the total die area of an FPGA [1]. Testing this amount of interconnect requires a highly scalable and portable approach.

To provide the scaling and portability needed to test the routing fabric for stuck-at faults, CIFT uses an algorithmic approach to identifying and then targeting PIPs in a device until all reachable PIPs are tested. CIFT makes use of a

The Waypoint Router uses an iterative algorithm of repeatedly finding a path from an existing route through a desired PIP and then repeating with a new PIP. Each test case thus consists of a single path that snakes through the device covering potentially hundreds to thousands of PIPs as seen in Figure 9. Using a single route in each test case reduces complexity, congestion and resource overhead associated with connecting multiple route end points back the harness. The route on each end connects to a small test harness with a state machine controlling the test including driving the test route with both logic 0 and logic 1 to check for respective stuck-at faults and interpreting the result.

Within CIFT, a dispatcher tracks the PIPs in the device and removes any PIPs that the Waypoint Router has already routed through. The dispatcher is then responsible for identifying the next PIP for the Waypoint Router to target from the remaining untested PIP. If the Waypoint Router is unable to route the PIP due to congestion, the dispatcher will move the PIP to the end of the queue to be tested in another bitstream. To reduce build time, the Waypoint Router does not perform any congestion negotiation such as rip up and reroute like other FPGA routing algorithms.

When choosing the next PIP to attempt, the dispatcher selects PIPs near the existing test route to reduce frequent long paths zigzagging back and forth through the device which would yield high congestion and few newly targeted PIPs. The dispatcher also uses special rules to encourage PIPs that can be quickly routed through to reduce build time.

Because the Waypoint Router can target PIPs all throughout the device, the number of bitstreams grows relative to the number of PIPs in a switchbox instead of the total number of PIPs in a device. There can be thousands of PIPs in a switchbox in FPGAs and, through experimentation, we have found that the Waypoint Router can target 2-4 PIPs in each switchbox. With over 3,000 PIPs in switchboxes in some FPGA families, this leads to potentially a couple thousand bitstreams to test 60-80% of the routing interconnect on the device; some PIPs are not targeted due to being in hard to reach locations such as next to hard IP like BRAMs, DSPs.

The algorithmic approach to testing interconnect employed by CIFT using the Waypoint Router is naturally flexible and can easily scale to different sizes of devices. Additionally, with only minor modifications, primarily consisting of defining routing paths through the slices, the Waypoint Router is highly adaptable to new device families and all supported Xilinx families can be targeted with just the two existing implementations of the algorithm. The Waypoint Router does require high visibility into the routing fabric and due to limited

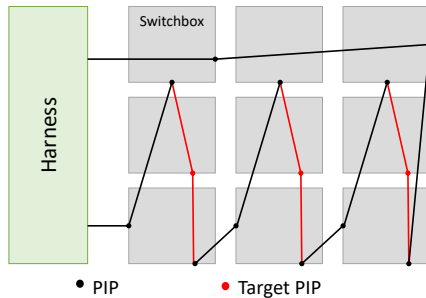


Fig. 9. CIFT Interconnect Test Structure

TABLE II
SAMPLE BRAM TEST METRICS

Family	Device	# Tests	# Bitstreams	Test Time
Xilinx Kintex 7	xc7k70t	24	25	Measure
Microsemi SmartFusion	To choose	Numbers coming		Measure
Microsemi PolarFire	To choose	Numbers coming		Measure

TABLE III
SAMPLE INTERCONNECT METRICS

Family	Device	# Bitstreams	Coverage	Test Time
Xilinx Kintex 7	xc7k70t ¹	1389	70.4%	Measure

visibility is not supported for Microsemi families.

IV. EXTENDING CIFT

A. New Families Within Supported Vendors

Porting CIFT to support a new family involves leveraging existing test infrastructure in CIFT to create test cases to cover the desired components of a device. The first step in porting is to identify structures in the FPGA to be tested. For PLBs, this includes the logic components such as LUTs, FFs and arithmetic units as well as routing muxes and data paths. For BRAMs, this usually involves identifying the possible configurations and shapes for the block. Especially for data paths through the PLBs there can be multiple options for how to test them. When identifying structures and data paths to test, it is helpful to match them to existing test cases in other families to allow as much reuse as possible. For example, a common step when porting PLB tests is to identify the LUTs and determine how previous LUT tests may be reused or modified to test the resources. Also important is identifying new resources with no parallels in existing test cases and will need test cases created from scratch for them. As support for families within a vendor increases, the frequency of finding previously unseen features decreases substantially.

With test cases, datapaths and TPGs selected for the tests, the next step to implementing tests is to create the descriptions of the UUTs. The descriptions are usually implemented in HDL with heavy use of placement constraints to enforce placement. For example, for Xilinx FPGA families, descriptions are created using the UNISIM library primitives with BEL, LOC, and RLOC constraints. In some cases, external EDA tools like TORC are needed to force routes that are not naturally used by the EDA tools.

B. Porting to New Vendor/EDA tools

Though CIFT provides a high degree of flexibility, supporting a new FPGA vendor requires a significant upfront cost. The task involves interfacing CIFT with EDA tools to perform synthesis, implementation, and bitstream generation of the test designs, programmatically finding the resources in a device, and identifying and implementing techniques to force the placement of the resources to test. Once performed for one

family, the capabilities can frequently be reused across many families of FPGAs. For example, we are able to use the same interface to Xilinx Vivado EDA tools to test all Xilinx Series 7, Ultrascale, and Ultrascale+ devices with minimal changes.

To perform implementation, CIFT provides interfaces with synthesize, place, route and generate bitstream methods. When supporting a new EDA flow, this interface will be extended with the corresponding EDA tool executed in the step. For Xilinx ISE, this involves executing the binaries for each step; for Xilinx Vivado, a single instance of Vivado is opened through which TCL commands are executed for each step.

CIFT also requires access to all of the resources it will target on the each device. The challenge getting this information may vary based on the EDA tool. For Xilinx ISE and Vivado, this information is quickly ascertained through the XDLRC and TCL interface respectively. For Microsemi, this information is found by searching the device in a process similar to fuzzing. For some families, not all information may not be readily attainable such as the routing interconnect in Microsemi.

The next requirement is creating the test circuits. As previously described, the circuits are created through a collection of HDL-described circuits, placement constraints, and occasionally manual physical manipulations where needed. Each EDA tool has its own method for constraining devices and when first targeting a new EDA tool, methods for programmatically creating the constraints need to be created. For Xilinx ISE, these constraints are created inside the HDL; for Vivado and Microsemi, the constraints are attached to the instances via TCL commands.

Last, for routing interconnect tests, either the Waypoint Routing algorithm must be implemented to work with the new EDA flow or the EDA flow must be integrated into an existing implementation of the Waypoint Routing algorithm. Implementing the Waypoint Routing algorithm can be difficult because, as with other routing algorithms, there are many design rules for a device that need to be controlled. Further, the appropriate tools, e.g. TORC, RapidWright, must exist for creating placed and routed netlists.

C. Case Study: Intel Quartus Pro

The CIFT project continues to support new families of FPGAs and fault models including porting CIFT to Intel family devices. This process makes heavy use of the existing CIFT infrastructure while requiring developing new techniques

¹Built on 25% of the device. Coverage is 70.4% of the PIPs in the targeted quarter of the device.

to interface the Quartus Prime EDA tools. We walk through the process of adding support for Intel devices as a case study of the approaches and challenges for porting CIFT to a EDA tool. Testing includes LABs – i.e. PLBs – BRAMs, and routing interconnect.

CIFT interfaces with the Quartus Prime Pro EDA tools similarly to Xilinx ISE. It primarily involves executing the individual binaries for each step of the FPGA EDA flow - synthesis, fitting (placement and routing), bitstream generation, and programming. However, Quartus does not provide as powerful of a query-able device interface as Xilinx ISE or Vivado. The approach taken is to generate a query-able device interface for a family and device the first time it is needed. This database is generated by opening a single instance of Quartus and executing a series of TCL commands to generate a textual device database files describing a family and the devices; this is somewhat similar to the approach used for Xilinx Vivado. The family database describes all types of blocks and sub-blocks that occur in the family of FPGAs. It also lists the types of routing resources that appear in the family as well as the devices and packages that make up the family. Individual device databases describe how blocks are laid out in an individual device. CIFT queries these family and device databases when generating tests.

In creating CIFT tests for LABs and BRAMs, CIFT draws heavily from existing TPGs. For BRAMs, the march pattern TPGs are reused with a wrapper to adapt the interface to match the M20K blocks. PLB tests are more difficult as the LAB structure has many significant differences compared to the Xilinx LUT structure. Nonetheless, many of the TPGs like the LUT TPG are easily adaptable to the LABs. UUT circuits are created by applying location constraints to force the LUTs and FFs as desired and customizing the HDL to force usage of the desired resource to test.

There are two primary challenges to creating routing interconnect tests for Quartus Prime Pro FPGA families. First, Quartus Prime Pro provides limited visibility into the routing interconnect. Unlike Xilinx EDA tools, a user cannot query the tools to find out what downhill or uphill PIPs exist from any given wire. To solve this challenge, CIFT must generate a routing databases for Intel families. This is possible by utilizing Quartus' routing constraint file (RCF) capabilities. A large number designs must be generated to use and expose all of the PIPs that exist in a device. By generating RCFs for these designs, a routing database describing all the PIPs in a device can eventually be built up and then used.

The second challenge is implementing the WayPoint Router. The overall approach remains the same as described in Section III. The part of the dispatcher that is used to track and decide which PIPs an individual test bitstream should target is largely unaltered from previous implementations. In order to implement the WayPoint Routing algorithm, an EDA tool like the Torc and RapidWright tools for Xilinx FPGAs is needed. To address this, The CIFT team has begun extending the open-source placement and routing tool, NextPnR, with partial support for Intel FPGA families. This new support

includes architectural support for supported Intel families and the ability to export a specially-crafted designs as a set of three files: a Verilog Quartus Mapping (VQM) file, a Quartus Settings file (QSF) containing placement constraints, and an RCF file. These files can be imported back into Quartus for bitstream generation. This has enabled an initial version the WayPoint Router to be implemented in NextPnR, currently supporting only a subset of Intel FPGAs.

While the WayPoint Router for Intel FPGAs is still under development, initial impressions are that it performs and will scale similarly to previous implementations. However, importing the custom-built test designs back into Quartus for bitstream generation is emerging as a likely bottleneck. In particular, processing an RCF and finishing the routing in Quartus appears to take up a large percentage of time. The current approach still relies on Quartus to complete routing for the test harness; it is possible that utilizing an RCF describing all the routes in a design may improve routing performance.

V. CONCLUSION

In this paper, we discussed the importance of independent functional testing and demonstrated how CIFT provides such testing for FPGAs. Further, the algorithms and infrastructure in CIFT are highly flexible and scalable and allow for relatively quick targeting of FPGAs. This is demonstrated by the many FPGA families and devices from multiple vendors that CIFT provides functional testing for. We also demonstrated how CIFT may be ported to support functional testing for Intel FPGAs.

Altogether, CIFT's scalability and flexibility make it a strong platform to develop further independent functional testing upon. This includes supporting new FPGA devices and manufacturers. CIFT can also integrate in new testing capabilities such as delay testing and detecting shorts in the routing fabric. The recent interest in eFPGA is also an area where CIFT may be able to contribute.

ACKNOWLEDGEMENT

This material is based research sponsored by the OSD Trusted and Assured Microelectronics (T&AM) Program under agreement number N00164-19-9-0001. The views, and conclusions contained herein are those of the authors and should not be interpreted as necessarily representing the official policies or endorsements, either expressed or implied, of the U.S. Government.

REFERENCES

- [1] A. Ye and J. Rose, "Using Bus-Based Connections to Improve Field-Programmable Gate-Array Density for Implementing Datapath Circuits," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 14, no. 5, p. 462–473, May 2006. [Online]. Available: <https://doi.org/10.1109/TVLSI.2006.876095>
- [2] N. Steiner, A. Wood, H. Shojaei, J. Couch, P. Athanas, and M. French, "Torc: Towards an Open-Source Tool Flow," in *Proceedings of the 19th ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, ser. FPGA '11. New York, NY, USA: ACM, 2011, pp. 41–44. [Online]. Available: <http://doi.acm.org/10.1145/1950413.1950425>
- [3] C. Lavin and A. Kaviani, "RapidWright: Enabling Custom Crafted Implementations for FPGAs," in *2018 IEEE 26th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2018, pp. 133–140.